

Bornal

DIY krux integration tests

qlrd - Krux contributor

Idea summary

Bitcoin Test Framework(s) reflections and practices.

Definitions [1]

Definitions by obj, POV and beneficiary

[2] names **what** is tested:

Definition 1.

The functional [tests] test the RPCs.

[3] names **who is looking**:

Definition 2.

(...) it takes pretty long. In general they take longer than unit tests. (...) pay some attention to how you write the tests and how many you write.

Neither used definitions names a beneficiary.

Candidates

Reference itself [4] (oop approach)

```
class MyTest(BitcoinTestFramework):  
  
    def set_test_params(self):  
        self.num_nodes = 2  
  
    def run_test(self):  
        self.connect_nodes(0, 1)  
        self.generate(self.nodes[0], 1)  
        self.sync_all()
```

C-lightning [5] (fixture and yield with core)

```
@pytest.fixture
def bitcoind(request, directory, teardown_checks):...

@pytest.fixture
def node_factory(request, directory, bitcoind, ...):
    nf = NodeFactory(request, test_name, bitcoind, ...)
    yield nf
    nf.killall([not n.may_fail for n in nf.nodes])

def test_pay(node_factory, bitcoind):...
```

Liana [6] fixture and yield with core + electrs + liana

```
@pytest.fixture
def bitcoind(directory):
    bitcoind = Bitcoind(bitcoin_dir=directory / "bitcoind")
    bitcoind.startup()
    bitcoind.rpc.generatetoaddress(101, ...)
    yield bitcoind
    bitcoind.cleanup()
```

```
@pytest.fixture
def lianad(bitcoin_backend, directory): ...
```

Floresta [7] Based fixture and yield with core + utreexod + floresta

```
@pytest.fixture
def node_manager(setup_logging, request):
    manager = FlorestaTestFramework(...)
    yield manager
    manager.stop()

@pytest.fixture
def bitcoind_node(node_manager) -> Node:
    node = node_manager.add_node_default_args(
        variant=NodeType.BITCOIND)
    node_manager.run_node(node)
    return node
```

See it from outside...

	Core	C-lightning	Liana	Floresta
lineage	itself	itself	C-lightning	Core ¹
who runs it	main()	pytest fixtures	pytest fixtures	pytest fixtures
binaries	the tree	PATH	*_PATH env	prepare.sh
RPC	authproxy	__getattr__ proxy	__getattr__ proxy	node.rpc
daemons	bitcoind	bitcoind, lightningd	bitcoind, electrs, lianaad	bitcoind, utreexod, florestad
scope	one script	function	function	function, class

¹It started with similar Core version

Challenges

1. Build
2. Dialect
3. Lifecycle
4. Synchronization
5. Oracle
6. Attribution
7. Version matrix

bornal [8]

Test the test

```
# install
```

```
$> uv add --dev git+https://github.com/qlrd/bornal.git@v0.0.1
```

```
# check the help
```

```
$> uv run bornal -h
```

```
# scaffold a template test
```

```
$> uv run bornal create the_test --template wallet-roundtrip
```

```
# first time: minutes of cmake
```

```
$> uv run pytest --build-bitcoin latest \  
                --wallet \  
                tests/integration/test_the_test.py
```

```
# next time it will be faster
```

```
$> uv run pytest tests/integration/test_the_test.py
```

```
# entry point for the plugin system
[project.entry-points."bornal.daemons"]
bitcoin-core = "bornal.plugins.bitcoind:CoreCompiler"
electrs      = "bornal.plugins.electrs:ElectrsCompiler"

# Access clients and daemons through `integration_test` fixture.
[project.entry-points.pytest11]
bornal = "bornal.fixtures"
```

1. Compiler → **build**;
2. Daemon → **lifecycle**;
3. Client → **dialect**;
4. entry points → **ownership**;
5. helpers that name the node → **attribution**;
6. the watch-only rule → **verification**.

Approach

```
@pytest.fixture(scope="module")
def test_factory(request):
    pytest.fail() # SCOPE IT YOURSELF

@pytest.fixture(scope="module")
def integration_test(request, test_factory):
    test = test_factory(data_dir=(...),...) # prev `BaseTest`
instance
    test._on_set_test_params() # runs YOURS `set_test_params` hook
    try:
        test._on_run_test() # runs YOURS `run_test` hook
        yield test
    finally:
        test._on_stop_test() # runs YOURS `stop_test` hook
```

Example of conftest.py

N = 2

```
class BaseTest(IntegrationTest):
    def set_test_params(self):
        # "bitcoin-core", "electrs", "utreexo", "floresta",
        "liana", etc.
        for _ in range(N):
            self.add_backend(<impl>, <daemon-args>=...)
            ...

    def run_test(self):
        self.log.info("Example test running") # do anything
        ...
```

```
@pytest.fixture(scope="module")
def test_factory(request):
    return BaseTest # bernal will call an instance of this class

# define abstract and consensual names, it's easier
@pytest.fixture
def alice(integration_test):
    return integration_test.backends[0]

@pytest.fixture
def bob(integration_test):
    return integration_test.backends[1]
```

Example of a test_the_test.py

```
from bornal.plugins.bitcoind import UNSPENDABLE_ADDRESS
from bornal.fixtures import (
    assert_block_count,
    connect_p2p,
    generate_to_address,
    sync_blocks # bornal has more in its bag
)
```

```
# Both nodes have already booted
def test_000_connect_p2p(alice, bob):
    connect_p2p(alice, bob)

# Nodes are still up and connected, so the order matters
def test_001_generate(alice):
    generate_to_address(alice, UNSPENDABLE_ADDRESS, 1)

# After yielding to ALL tests, the nodes will stop.
def test_002_sync_blocks(alice, bob):
    sync_blocks(alice, bob)
    assert_block_count(bob, 1)
```

WIP with bornal: Krux integration tests

krux-integration-tests [9]

- two bitcoin-core nodes: one **coordinates**, the other **witnesses**;
- Core imports the tpub, funds it, builds the unsigned PSBT [10];
- the Python modules of krux/embit inspect and sign;
- Core finalizes and broadcasts; the witness should see the tx;
- Krux must refuse: wrong purpose, wrong network, wrong policy;
- and malformed PSBTs, fabricated previous txs, non-standard sighashes, outputs above inputs.

1. **logging**
2. **persistence**
3. **fidelity**

Plans

To solve

- **lifecycle**
- **oracle**
- **drift**
- **plugins**
- **signer warnet**

Bibliography

Bibliography

- [1] qlrd, “Bitcoin Test Framework (vintenum's study hour).” [Online]. Available: <https://github.com/qlrd/bitcoin-test-framework-presentation>
- [2] A. Chown, “How does bitcoin functional test framework work?” [Online]. Available: <https://bitcoin.stackexchange.com/questions/73932/how-does-bitcoin-functional-test-framework-work>
- [3] F. Jahr, B. Bishop, and M. Folkson, “Bitcoin Core Functional Test Framework.” [Online]. Available: <https://btctranscripts.com/edgedevplus/2019/bitcoin-core-functional-test-framework>
- [4] The Bitcoin Core developers, “test/functional/test_framework.” [Online]. Available: https://github.com/bitcoin/bitcoin/tree/master/test/functional/test_framework
- [5] Core Lightning developers, “pyln-testing: Core Lightning's test framework.” [Online]. Available: <https://github.com/ElementsProject/lightning/tree/master/contrib/pyln-testing>

Bibliography (ii)

- [6] Wizardsardine, “Liana functional tests (test_framework).” [Online]. Available: https://github.com/wizardsardine/liana/tree/master/tests/test_framework
- [7] The Floresta developers, “Floresta functional tests.” [Online]. Available: <https://github.com/getfloresta/Floresta/tree/master/tests>
- [8] qlrd, “bortal: kit for bitcoin related integration test tools.” [Online]. Available: <https://github.com/qlrd/bortal>
- [9] qlrd, “krux-integration-tests.” [Online]. Available: <https://github.com/qlrd/krux-integration-tests>
- [10] A. Chow, “BIP 174: Partially Signed Bitcoin Transaction Format.” [Online]. Available: <https://github.com/bitcoin/bips/blob/master/bip-0174.mediawiki>

Bibliography (iii)

- [11] Cangaçologia, “Há oitenta e cinco anos morreu o cangaceiro Corisco, o Diabo Loiro.” [Online]. Available: https://www.instagram.com/cangacologia/p/DKFXBAox38u/?img_index=1
- [12] Wikipedia contributors, “Cangaço.” [Online]. Available: <https://en.wikipedia.org/wiki/Canga%C3%A7o>

Appendix I: “Cangaceiro” with bornals [11]



Figure 1: “Cangaceiro” with bornals carrying a variety of tools and meals, to survive the aridness and a hard way of life [12].